

Nova Strike — technical documentation

Review Build 1 (baseline — draft) · build 0.1.0+0c2fc79 · written by the Builder from the code at that commit. Builder-reported: this describes the design of the code; the tests listed at the end are what checks it.

1. Engine and targets

- **Godot 4.7.2**, GL Compatibility renderer, GDScript with explicit types.
- Portrait. A fixed **1080 × 1920 logical play area** (`core/playfield.gd`) is scaled uniformly on every screen shape; extra space on tall phones, tablets and desktops is background art only, so every player sees the same encounter.
- Builds: **Web** (single-threaded, desktop-class texture compression), **Android APK** (ETC2/ASTC, signed with the studio key), **Android Benchmark** APK (plays a fixed scene with the bot and uploads frame times; used by Firebase Test Lab). iOS is planned through the web build now and TestFlight later.
- Languages: **English and Arabic** (right-to-left layouts, Arabic font Tajawal). Every string exists in both (`i18n/ui.csv`, `combat.csv`, `content.csv`).

2. Structure

Area	Where	What it does
Shared kit	<code>core/</code>	Config and build ID, settings save, localisation, audio buses, router, UI helpers, analytics, cloud, perf recorder, benchmark mode, dev panel (dev builds only)
Profile	<code>game/profile.gd</code> (autoload Profile)	Everything the player earns: credits, hardware tiers, missions, records, mastery, skins
Session	<code>game/game_state.gd</code> (autoload Game)	Which mode/mission/unit is being played; hands the result to Profile exactly once
Data	<code>game/data/units.gd</code> , <code>missions.gd</code> , <code>enemies.gd</code> , <code>bosses.gd</code>	3 units × 6 hardware families × 4 tiers; 48 missions; 13 enemy roles; 6 bosses
Aircraft (3D)	<code>game/units/</code>	One rig per aircraft (Striker, Guardian, Arcanist) used by Home, Hangar, Store, Briefing and combat
Combat	<code>game/combat/</code> , <code>scenes/combat.tscn</code>	Player, weapons, temporary levels L1–L4, Tactical, Bomb, Super, enemies, bosses, bullets, pickups, HUD, pause/defeat
Screens	<code>game/ui/</code> , <code>scenes/*.tscn</code>	Title, Home, Hangar, Campaign, Briefing, Results, Store, Settings, Endless, Mastery, Training, Ending, Credits
Assets	<code>assets/</code>	3D units (GLB), enemy/boss/environment/pilot/UI art, music and sound
Tests	<code>tests/</code>	Automated suites, capture scenes, the bot journey (<code>demo_run</code>), review recordings (<code>tests/review/</code>)

Flow: Title → Home → Hangar / Campaign → Briefing → Combat → Results → next mission; Endless → Combat → Results; Training → Combat → Home; mission 6.08 victory → Ending → Credits.

3. Aircraft

- Each aircraft is a generated 3D model (Rodin Gen-2.5 High, from approved canon views) cut into named moving parts by scripts (`pipeline/rig_*.py`); nothing is hand-edited, so it can be rebuilt.
- **One rig per aircraft everywhere.** The Hangar, Home and combat all call `UnitFactory.make(unit_id)`, so a part bought in the Hangar is the same mesh on the same mount in combat.
- Hardware families: primary, side, ordnance, defense, engine, reactor (tiers 0–4). Side, ordnance and defense add parts at every tier; primary, engine and reactor change shape at the first tiers and mostly by finish after that (known gap).
- Combat view: the rig renders into its own transparent 3D layer placed inside the 2D play area; the reactor is exactly the hitbox. On phones the 3D layer renders without MSAA at up to 0.7 of the logical size (performance mode).
- Super: each aircraft has a mechanical transformation (parts unfold ~1.4–1.6× wider and close back exactly); combat applies the Super on top of the hardware snapshot.

4. Combat rules (order of effects)

Hardware snapshot (`UnitDefs.resolve`) → temporary level L1–L4 → Super multiplier → statuses → caps. A module is never applied twice. Missions are 10 authored waves (~90–120 s) shown on the slim right-edge rail; boss missions add a multi-phase boss. No popups interrupt play.

5. Saves

- `user://profile.json`, **schema v2**, written atomically (temp file, then replace) with a last-known-good backup (`profile.bak.json`) and recovery of an interrupted write. Version-1 saves are migrated.
- **Purchases** carry a transaction ID: pressing Install twice charges once; a crash mid-purchase can never charge twice.
- **Mission results** carry a result ID: a reward is granted once, however many times the screen is revisited.
- Settings are stored separately (`user://save.json`, same atomic/backup rules).
- Offline: the game never needs a connection to play or save.

6. Services

There is **no ad SDK and no store billing** in this build. The rewarded-ad buttons (×2 reward, one revive) and the Supporter pack show "not available in this test build". The logic behind them is tested with fixtures; nothing fakes a completed ad or purchase.

7. Audio

Music and sound effects on separate buses (Music, SFX) under Master, volumes from Settings. Sources: ElevenLabs sound effects and Stable Audio loops (provenance for every file in the audio manifest). **Web issue in this build:** the buses were created with `AudioServer.add_bus()`, which in Godot 4.7.2's browser audio puts the new bus in front of Master and routes Master into a loop, so browsers are silent. Fixed after this build by growing `AudioServer.bus_count` instead (Review Build 1a).

8. Build and delivery

Every push that changes the game runs the studio's cloud build: tests → captures (phone EN/AR, tablet, desktop) → the bot's gameplay video → web + signed APK → smoke test → publish to the builds site with `build.json` (build ID, links, tests). Review builds are pinned copies under `reviews/nova-strike/<slug>/`; review packages are made by the Brain's "Review package" workflow from `review/<slug>/package.json`.

9. Automated checks (re-run on this build's code)

test_runner 47/0 · combat_tests 35/0 · qa_suite 174/0 · ui_tests 0 failures · ui_flow 0 failures · content_validator PASS · units_check PASS · economy_sim PASS. Logs are in the package (`tests/`). These checks confirm behaviour and data; they don't judge how the game looks or feels.